# Building Reports, Dashboards, and Custom Pages (UI Builder) for the Application

*Vehicle Service & Repair Request Management — Reporting & Dashboard Use Case*

**Shrivarthini**
**241401100**

## 1. Aim

To build multiple report views — a Kanban report grouped by Service Status and a Calendar report for tracking service appointments — on top of the Service_Request form, and to design a custom Workshop Dashboard page using Zoho Creator's UI Builder (Page Builder) that gives the workshop's service-advisor team an at-a-glance view of total service requests, service-bay capacity utilised, and request volume by service type and by repair status.

This addresses the service advisors' need to move from a flat list of repair requests to visual, role-friendly views that make it easy to see which stage each vehicle is in, check the service schedule at a glance, and monitor overall workload trends without opening every record individually.

## 2. Procedure

**Step 1: Create the Kanban report** — A new report named Status_Board was added on the Service_Request form and its type was set to Kanban.

**Step 2: Configure Kanban properties** — In the Report Builder, Group Columns By was set to Service_Status so each column represents one stage of the repair workflow (Requested, Inspection, In Repair, Ready for Delivery), with the Empty Value Column and Deleted Value Column hidden and Record Count Per Column hidden for a clean board.

**Step 3: Create the Calendar report** — A second report, Service_Schedule, was added and its type was set to Calendar to visualise service appointments across a month/week/day grid.

**Step 4: Configure Calendar properties** — The Event title field was bound to Service_Type, and both Event starts on / Event ends on fields were bound to Service_Date, with the Default view set to Month and the week starting on Sunday.

**Step 5: Open the Page Builder** — A new custom page named Workshop Dashboard was created for the application using Zoho Creator's Page Builder (UI Builder), which exposes a palette of elements — Panel, Board, Chart, Gauge, Search, Form, Report, Snippets, Button, Widgets, and Viewer.

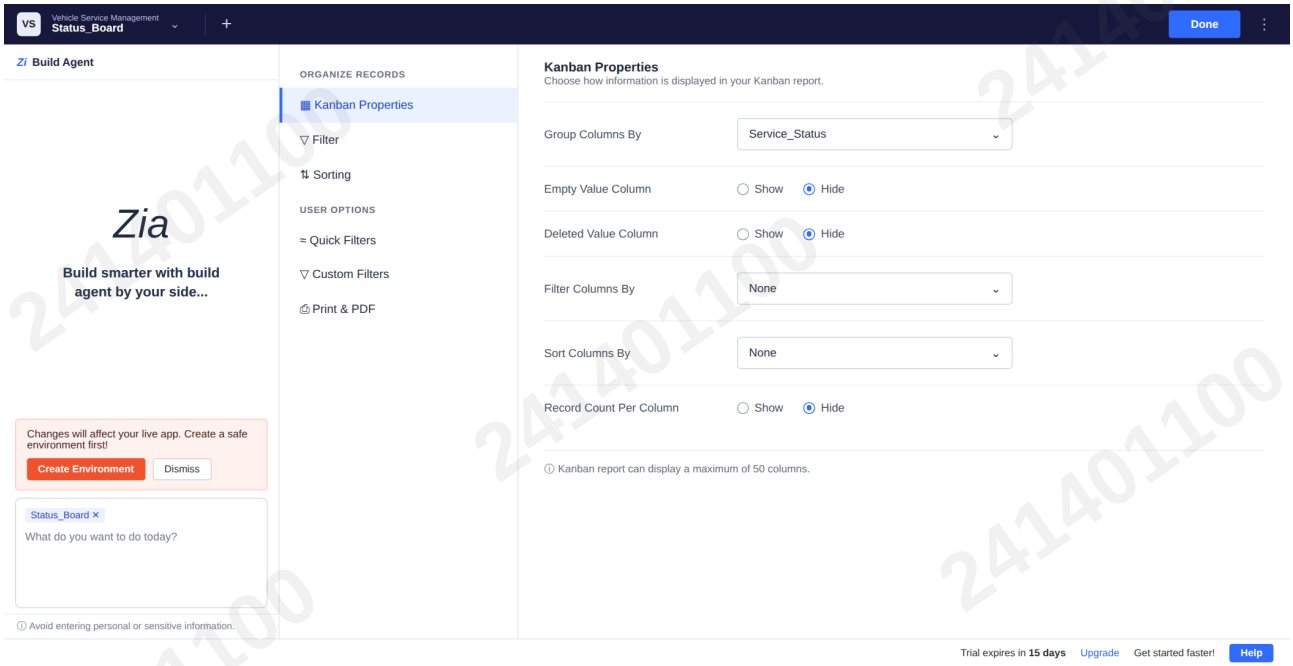
**Step 6: Embed the report grid** — A Report element titled All Service Requests was placed at the top of the page, listing every repair request in one place.

**Step 7: Add summary panels** — Two summary panels were added below the report — Total Service Requests, showing the live count of records, and Bay Capacity Utilised, showing the percentage of available service-bay slots filled.

**Step 8: Add chart widgets** — Two Chart elements were added at the bottom of the page: a Column chart plotting service requests by service type, and a Donut chart summarising requests proportionally by service status — giving the workshop team a quick visual breakdown alongside the numeric figures.

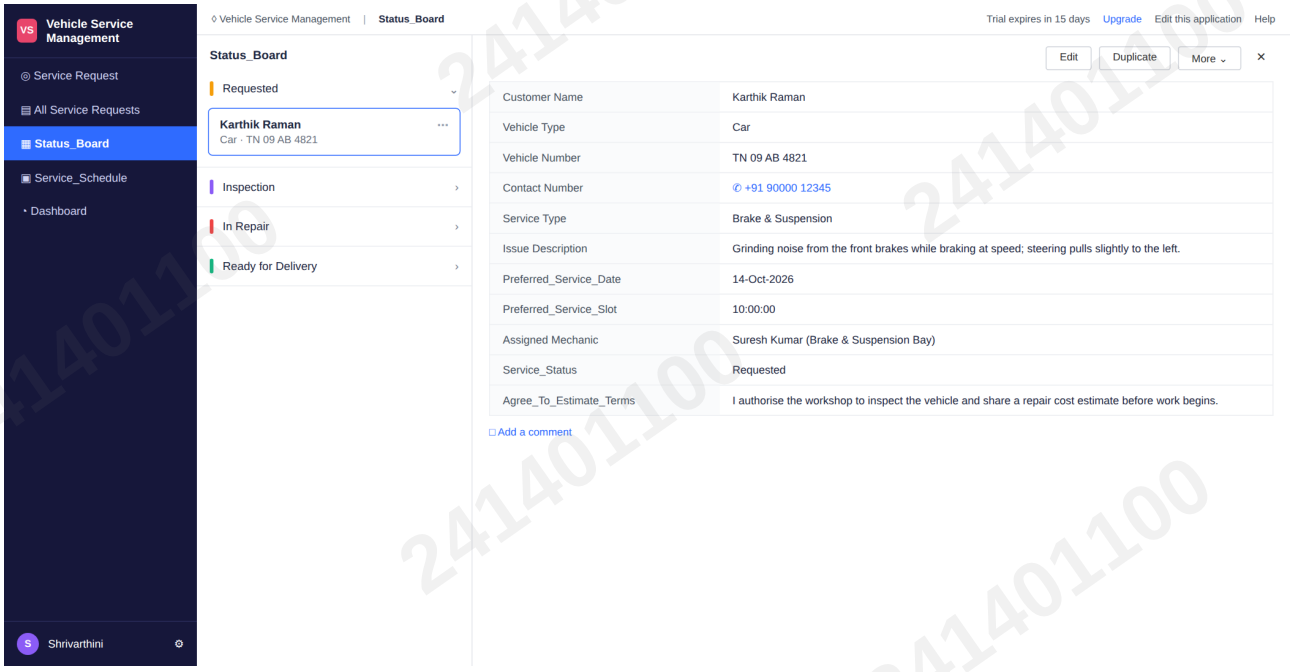
# 3. Output

## 3.1 Report Builder — Kanban Properties for "Status_Board"



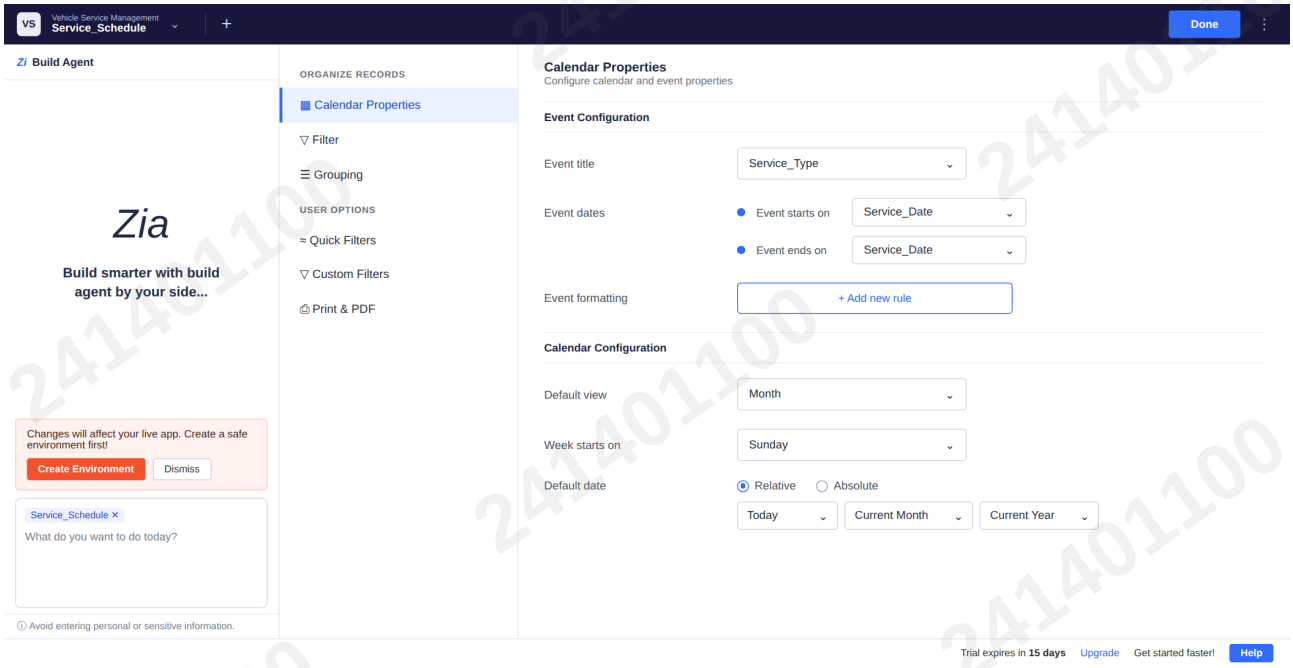

The Kanban Properties panel shows Group Columns By set to Service_Status, with the Empty Value Column, Deleted Value Column, and Record Count Per Column all set to Hide for a clean board. This is where the status-wise grouping used in the live Kanban report is configured.

## 3.2 Live Kanban report — "Status_Board"



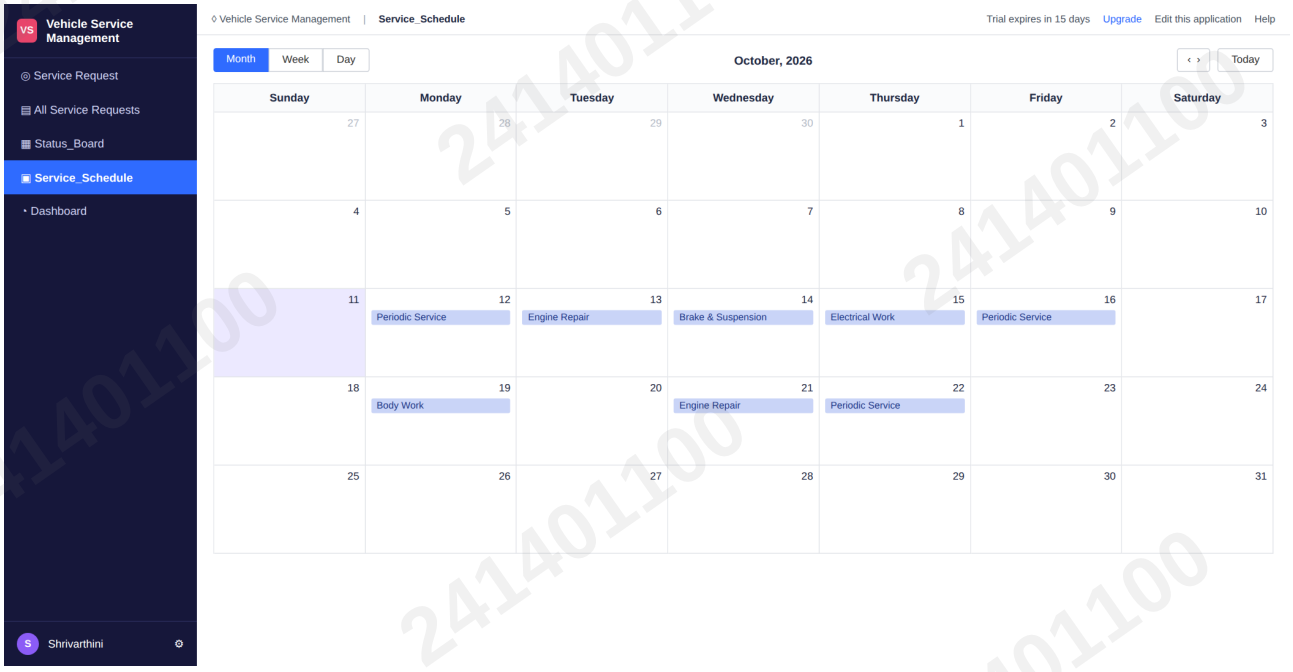

The published Status_Board report displays one column per repair stage, with each vehicle sitting under its current service status. Selecting a record opens the full request details — customer name, vehicle type and number, contact number, service type, issue description, preferred service date and slot, and assigned mechanic — confirming the grouping works as configured.

## 3.3 Report Builder — Calendar Properties for "Service_Schedule"



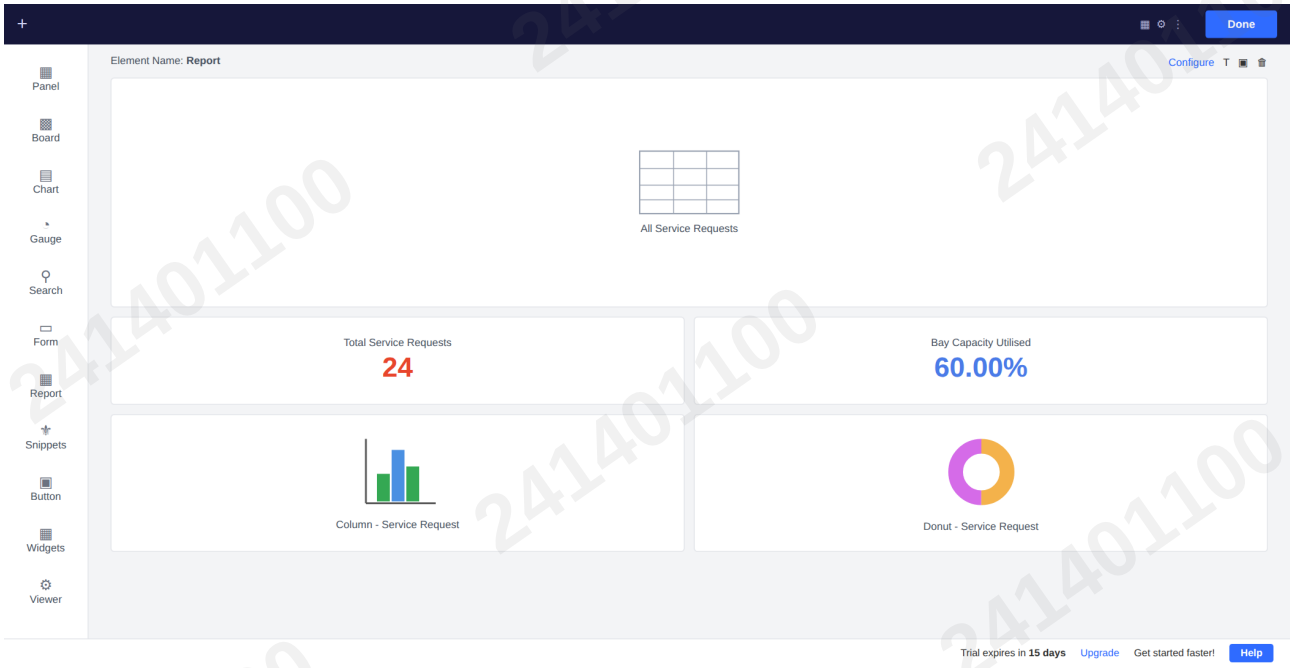

The Calendar Properties panel binds the Event title field to Service_Type, and both the Event starts on and Event ends on fields to Service_Date, with the Default view set to Month and the week starting on Sunday. This configuration drives how each service appointment is plotted on the calendar grid.

## 3.4 Live Calendar report — "Service_Schedule"



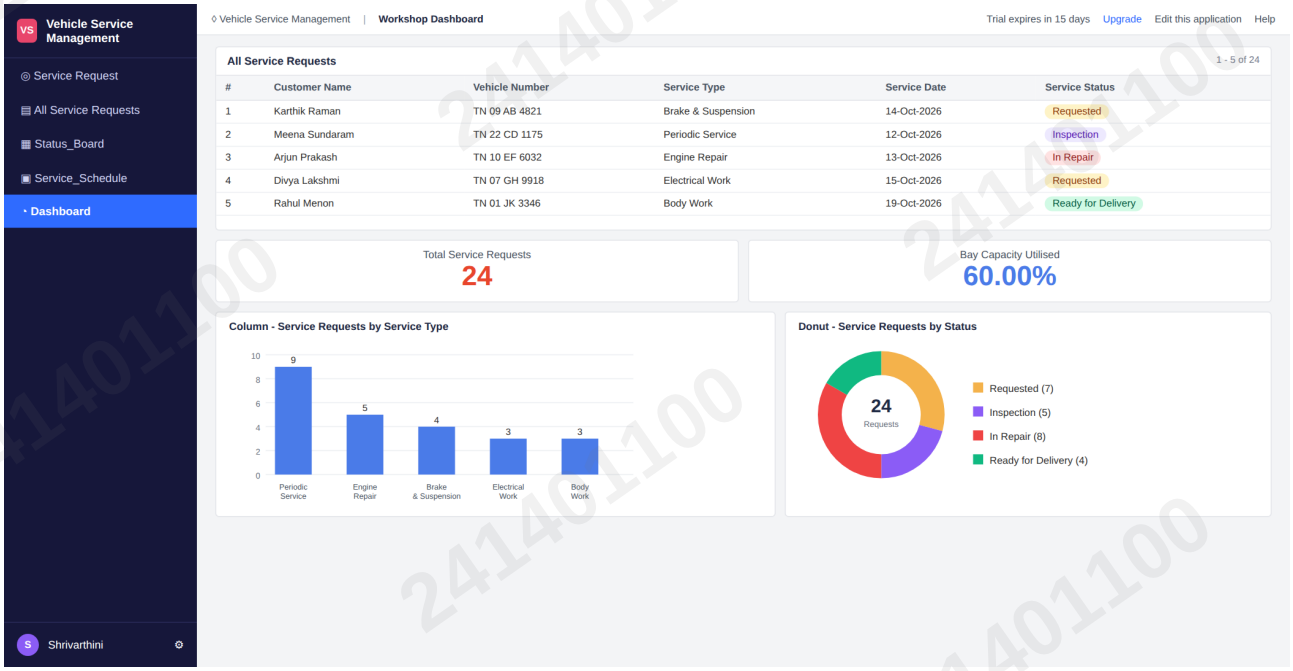

The published calendar shows the month in Month view, with each booked service type plotted directly on its service date. The sidebar also lists the other reports built on the form, including All Service Requests and Status_Board.

## 3.5 Page Builder (UI Builder) — Designing the Workshop Dashboard




The Page Builder view shows the widget palette (Panel, Board, Chart, Gauge, Search, Form, Report, Snippets, Button, Widgets, Viewer) on the left, and the Workshop Dashboard page under construction on the right: an embedded All Service Requests report grid at the top, followed by two summary panels — Total Service Requests and Bay Capacity Utilised — and two chart widgets, a Column chart and a Donut chart, both sourced from the Service_Request form.

## 3.6 Live custom page — "Workshop Dashboard"




The rendered Workshop Dashboard page pulls live data directly from the Service_Request form: the All Service Requests report lists every submitted record, the Total Service Requests panel shows a live count of records, and the Bay Capacity Utilised panel shows the percentage of service-bay slots filled. The Column and Donut charts break this same data down visually by service type and by status, giving the service team a single-screen summary of workshop activity.

## 4. Final Result

Two additional report views were successfully built on the Service_Request form: a Kanban report that groups repair requests by Service Status for quick stage-wise triage, and a Calendar report that plots every service appointment on its booked date for day-to-day workshop scheduling.

A custom Workshop Dashboard page was designed using Zoho Creator's UI Builder, combining an embedded report grid, two live summary panels — Total Service Requests and Bay Capacity Utilised — and a Column chart and a Donut chart into a single at-a-glance view of workshop activity across all service types and repair stages.

All views were verified in live/preview mode with sample data, confirming that the Kanban grouping, calendar date-binding, and dashboard widgets render correctly and pull live records from the Service_Request form.

The exercise met its objective: the workshop's service-advisor team now has three complementary, low-code views — grouped, scheduled, and summarised — built entirely without custom code, in place of scrolling through a single flat report.